

DESIGN AND IMPLEMENTATION OF A

Reconfigurable Dataflow Matrix Multiplication Accelerator

WEIGHT-STATIONARY SYSTOLIC ARRAY • FPGA-
VERIFIED

Team: Phani • Sanjay • Aditya • Amey | Advisors: Mr. Pushkar, Ms. Nandini

Final Project Presentation

02 Abstract

What we built

A parameterizable 4×4 weight-stationary systolic array accelerator for integer matrix multiplication, implemented in SystemVerilog and targeting Xilinx Zynq FPGAs.

How we verified it

Bottom-up, self-checking testbenches with golden-model scoreboards for every module; black-box top-level verification including deliberately poisoned padding data; synthesized and deployed on a real ZedBoard, driven live over UART from a host PC.

What it demonstrates

A real application (RGB image channel extraction), weight-stationary reuse for efficiency, and complete synthesis results — timing, area, and power — confirming the design works on real silicon, not just in simulation.

03 Table of Contents

01 Title Page

02 Abstract

03 Table of Contents

04 Introduction

05 Literature Survey

06 Architecture

07 Methodology

08 Objectives

09 Work Completion / Implementation Status

10 Results

11 Pros and Cons

12 Future Scope (incl. Applications)

13 Paper Publication

14 Conclusion

04 Introduction

Why matrix multiplication matters

- Matrix multiplication is the core operation behind neural network inference — every fully-connected and convolutional layer reduces to it.
- It also underlies image processing (filters, transforms), scientific computing, computer graphics, and countless other numerical workloads.
- The same basic operation — multiply and accumulate, repeated many times over a regular grid of data — shows up at every scale, from a single small filter to massive AI models.
- Because the pattern is so regular and so common, hardware specifically shaped around it can be dramatically more efficient than general-purpose computation.

PROJECT GOAL

Design, verify, and physically deploy a matrix multiplication accelerator — end-to-end, from RTL through simulation to real FPGA hardware.

04 Introduction

Why dedicated hardware, not a CPU

- General-purpose CPUs execute one instruction at a time (or a handful, with limited parallelism) — fundamentally mismatched to matrix multiplication's massive, regular parallelism.
- CPUs also spend significant time and energy repeatedly moving data between memory and the arithmetic unit — the same values often get re-read from memory over and over.
- A dedicated accelerator can instead keep data resident where it's needed and move only what must move, cycle by cycle, exploiting the operation's structure directly.

GENERAL-PURPOSE CPU

- Data repeatedly re-read from memory
- Limited parallel arithmetic per cycle
- Flexible, but inefficient for this specific pattern

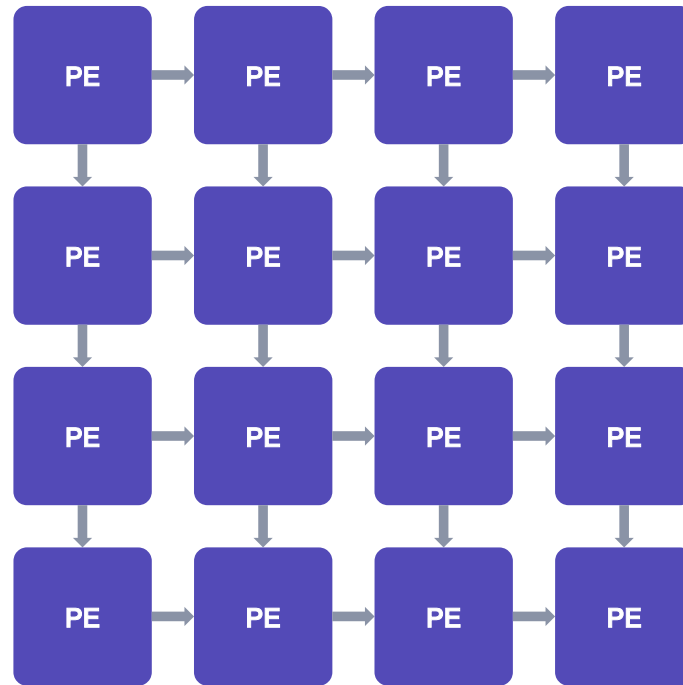
DEDICATED ACCELERATOR

- Data stays resident where it's reused
- Many multiply-accumulates happen every cycle, in parallel
- Purpose-built for exactly this operation's structure

04 Introduction

What is a systolic array?

A grid of simple processing elements passing data to their neighbors in a synchronized, wave-like pattern — reusing data across many computations without repeatedly re-reading memory.



- Each PE only ever talks to its immediate neighbors — no long wires, no central bottleneck.
- This project's array is exactly this shape: 4x4, matching the grid shown here.

05 Literature Survey

Part 1 of 2



Systolic Arrays at Scale

Google's TPU — the canonical example of systolic-array matrix multiplication hardware in production use. Demonstrates the same core weight-stationary principle this project implements, at industrial scale.

Confirms that the weight-stationary approach chosen for this project is not a toy simplification — it's the same fundamental idea behind real, deployed AI accelerators.



Alternative Dataflows

Eyeriss and its row-stationary dataflow — a useful contrast to this project's weight-stationary design choice, optimizing instead for reuse of partial sums and activations together.

Studying this alternative helped clarify exactly which tradeoffs weight-stationary design makes, and why it fits this project's goals.

05 Literature Survey

Part 2 of 2



Dataflow Taxonomy

The general classification of accelerator dataflows — weight-, output-, input-, and row-stationary — and where this project's weight-stationary choice fits within that landscape.

Understanding the full taxonomy made the design choice deliberate rather than arbitrary — weight-stationary was selected because weights are reused far more often than any other operand in this project's target workloads.



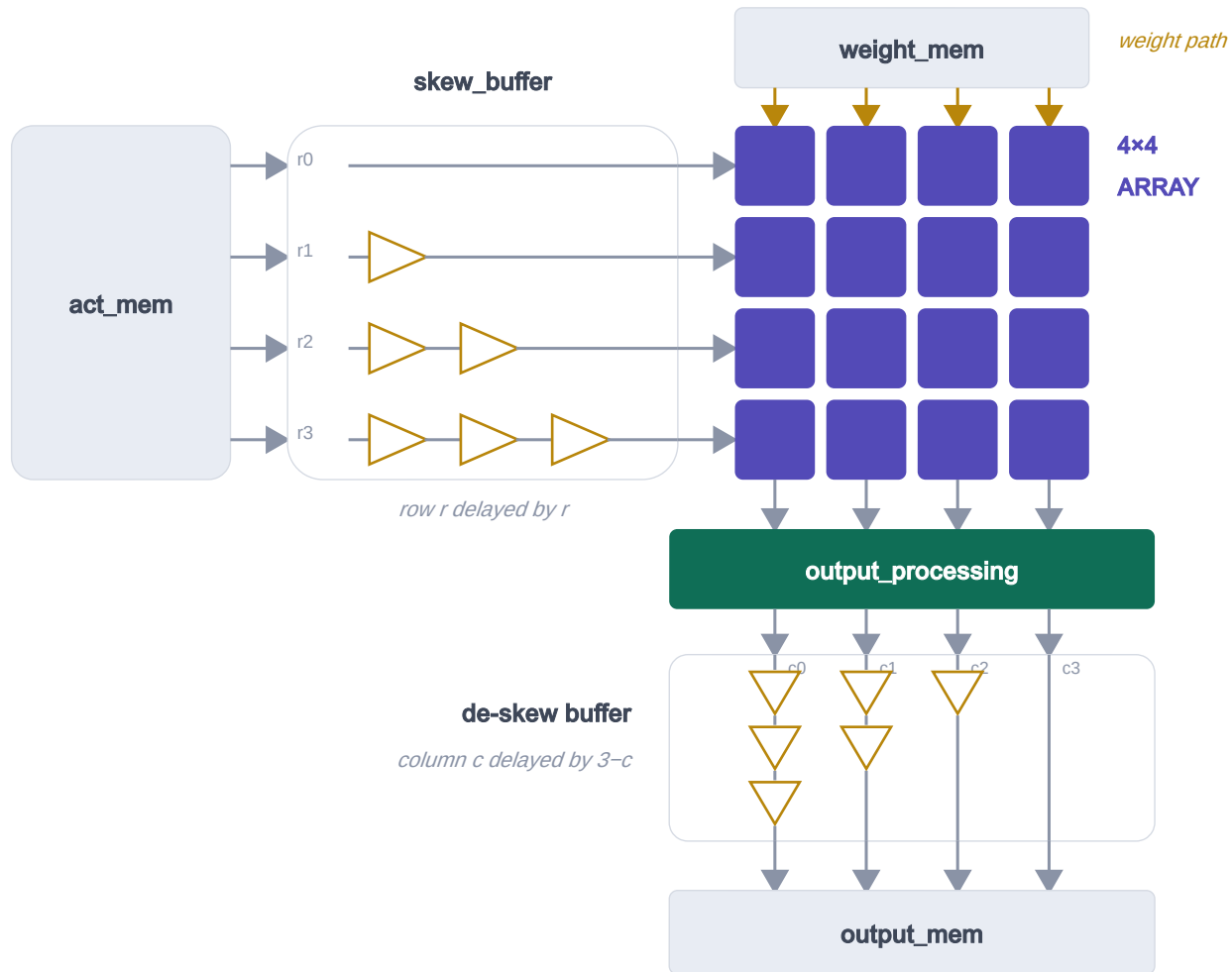
Reduced-Precision Arithmetic

int8 quantization as a common, practical choice in real inference accelerators — directly motivating this project's 8-bit input / 32-bit accumulator design.

The wide 32-bit accumulator, paired with narrow 8-bit inputs, mirrors the standard quantized-inference pattern used throughout the industry.

06 Architecture

Full system diagram — all connections



address_gen

drives read/write addresses and padding masks for weight_mem, act_mem and output_mem

control_unit

sequences LOAD → COMPUTE → DRAIN → DONE, drives the phase and the array's wload

Weight-stationary: weights enter the north edge once and stay resident in every PE. Activations stream in from the west and keep moving east. Partial sums fall south, one row per clock, so the result path — output processing, de-skew and output_mem — hangs below the array rather than beside it.

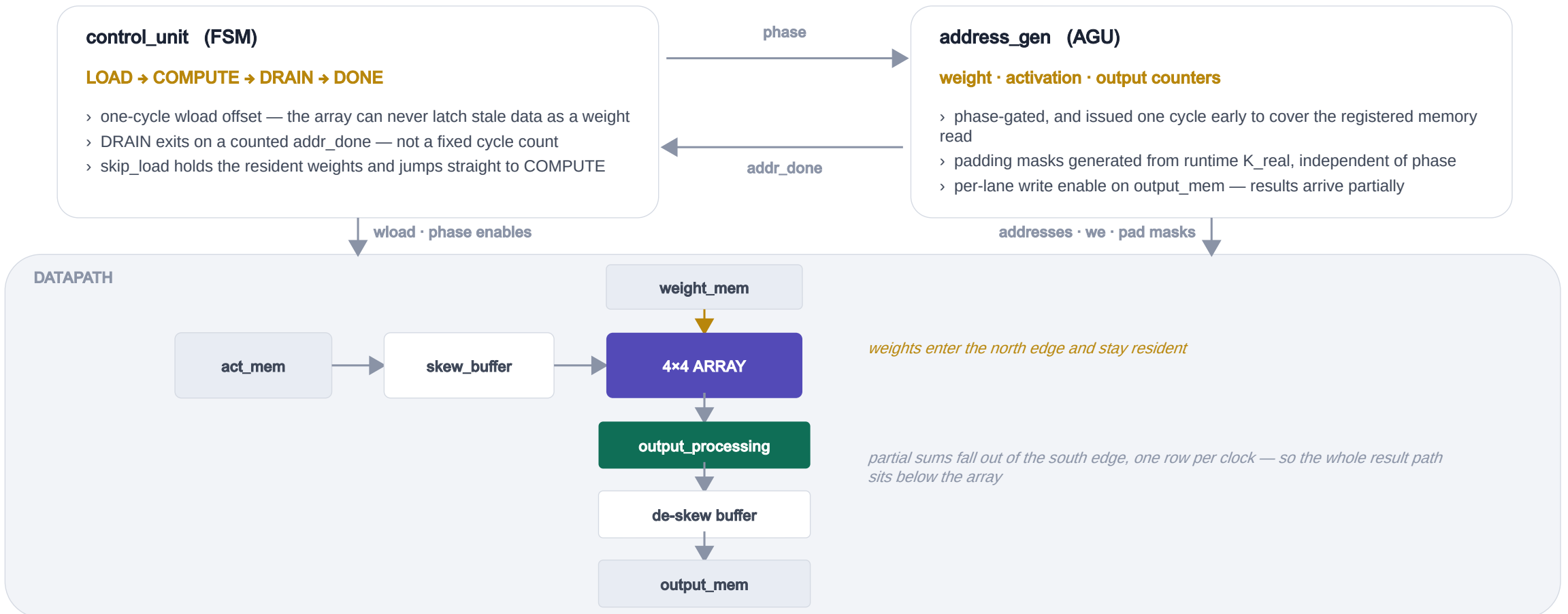
06 Architecture

Whole-system view — control unit and address generator

HOST (UART link)

start · K_real · skip_load · memory writes → busy · done · results

The FSM owns when — phase durations and wload timing. The AGU owns where — every address, write enable and padding mask.



06 Architecture

Module-by-module summary

Processing Element (PE)

Weight-stationary MAC. Signed 8-bit inputs, 32-bit accumulator. Reuses the partial-sum wire for weight loading — no separate weight-distribution network. Valid token propagates alongside data.

Systolic Array

4×4 grid of PEs, parameterized via generate loops, nearest-neighbor wiring only — no long wires, no central bottleneck.

Skew / De-skew Buffer

One reusable module (REVERSE parameter) — staggers activations in on the way in, realigns the diagonal output back into clean rows on the way out.

Memory Subsystem

Separate weight/activation/output memories. Row-at-a-time wide access, 1-cycle registered read, per-lane write-enable on output (results arrive partially, not all at once).

Output Processing

Optional bias + ReLU, always-on scale (shift) and saturate — converts 32-bit accumulator values down to 8-bit results.

Address Generator (AGU)

Three counters (weight/act/output), phase-gated, issue addresses one cycle early to match memory latency. Generates padding masks from runtime K_real, independent of phase.

Control Unit (FSM)

5-state FSM. Implements a critical one-cycle wload timing offset so the array never loads stale data as a weight. DRAIN exits on a real counted signal, not a fixed cycle count. Supports skip_load for weight reuse.

Padding

Runtime K_real input masks unused rows to zero at both input edges. Verified with deliberately non-zero (“poisoned”) padding data, not just zeros.

06 Architecture

Inside a processing element

One shared wire, no multiplexer — wload decides which register captures it.

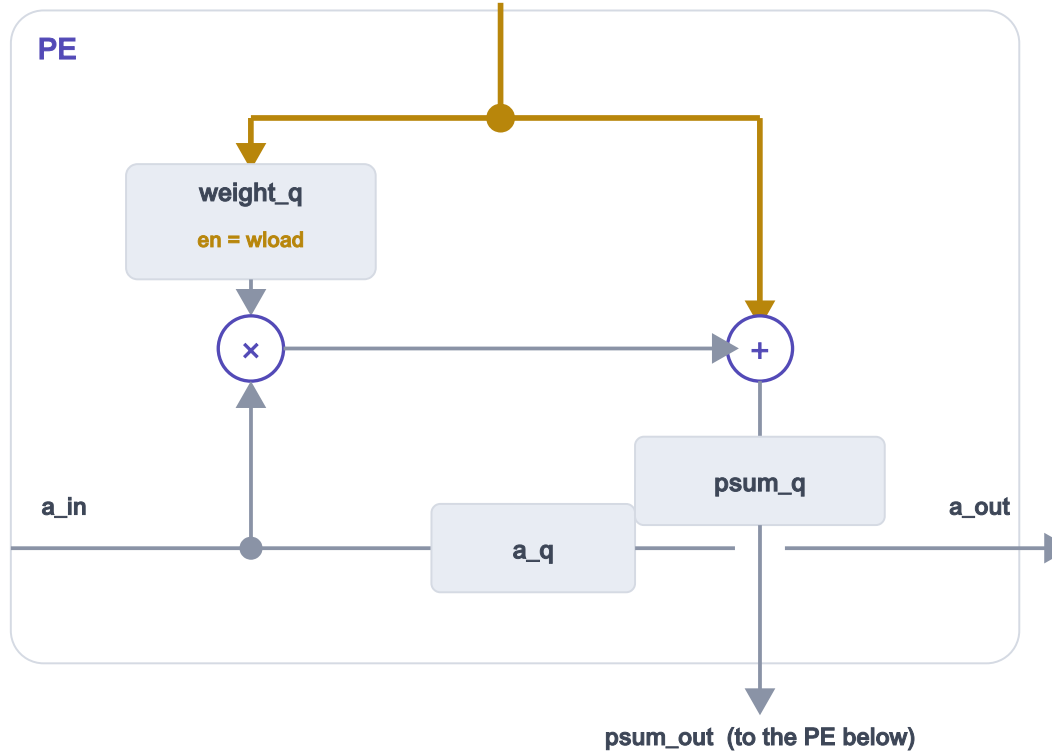
psum_in / weight_in
one shared 32-bit wire

wload = 1 LOAD

The wire carries a weight. Only weight_q has its enable asserted, so only weight_q captures it.

wload = 0 COMPUTE

The wire carries a partial sum. It goes straight into the adder; weight_q holds what it already has.



no select stage

Nothing on this path chooses between two 32-bit sources. The wire simply fans out to both destinations, and the enable on each register decides which one acts on it that cycle — 16 times over, once per PE.

- wload is a capture enable, not a data select — so no 32-bit select stage sits in the datapath of any of the 16 PEs, and no separate weight-distribution network is needed.

06 Architecture

Padding & poison-data verification

- Matrices smaller than 4×4 are supported via a runtime `K_real` input that masks unused rows to zero at the two input edges.
- The array always computes a full 4×4 result — unused output rows are simply never read back, with no extra output-side cleanup logic needed.
- `K_real` is a runtime input, not a compile-time parameter — the same synthesized hardware handles any matrix size from 1×1 up to 4×4 without rebuilding.

WHY “POISON” DATA, NOT JUST ZEROS

- If padding rows are pre-loaded with deliberately non-zero (“poisoned”) values instead of zeros, a correct result is only possible if the zero-injection hardware is genuinely working — not coincidentally passing.
- A bug that failed to mask padding would leak the poison values into the result — an unmistakable, easy-to-catch signal, unlike a zero-padding test which could pass by accident even with broken masking logic.
- This test ran across every `K_real` value from 1 to 4, confirming the masking logic is correct at every supported matrix size, not just the common cases.

06 Architecture

Dataflow comparison

Dataflow	Resident in PE	Streams through	Reuse achieved	Partial sums	Best fit
Weight-Stationary	One weight, held for the whole tile	Activations, west → east	Weight read once per tile	Move north → south, accumulate in flight	Inference with fixed trained weights
Output-Stationary	One accumulating partial sum	Weights and activations, both broadcast	Partial sum never leaves the PE	Stay in place until complete	Deep reduction, large K
Input-Stationary	One activation	Weights streamed past it	Activation read once	Move between PEs	Batched inference, reused feature maps
Row-Stationary	A row of weights and activations	Sliding feature-map rows	Balanced across all three operands	Accumulated along a PE row	Convolution with high filter reuse

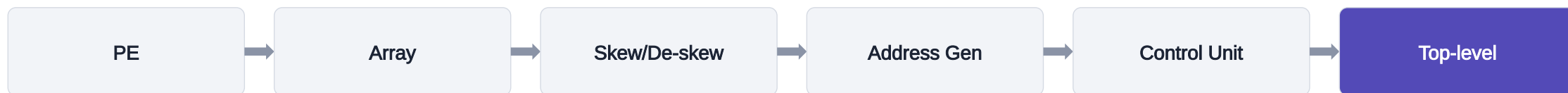
Why weight-stationary was selected. Trained weights are fixed at inference time, so they are the only operand guaranteed to be reused across the entire activation stream. Collapsing all weight-memory traffic into a single load phase removes it from the steady-state loop, keeps PE control to one enable bit, and reduces partial-sum movement to a short, regular nearest-neighbour hop.

07 Methodology

- 1 Architectural decisions — weight-stationary dataflow, PE design, array topology
- 2 Bottom-up RTL implementation, one module at a time
- 3 Self-checking testbench development per module, with golden-reference models
- 4 Integration and black-box top-level verification
- 5 Padding extension and poison-data verification
- 6 Weight-stationary reuse optimization
- 7 Application development — RGB channel extraction
- 8 Synthesis and real FPGA deployment

07 Methodology

Verification approach — bottom-up



- Each testbench is self-checking against a golden reference model, with a per-task pass/fail scoreboard — no manual waveform inspection required to confirm correctness.
- Tests specifically target failure modes likely to hide silently: signed-arithmetic errors, bubble/valid-gating gaps, reset incompleteness, and off-by-one timing — not just nominal-case functionality.

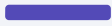
The top-level design is verified as a BLACK BOX — driven only through external ports, mirroring how a real host would use the accelerator, so a passing result reflects the system as a whole.

08 Objectives



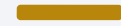
Design

A parameterizable, weight-stationary systolic array accelerator



Verify

Correctness exhaustively at both module and system level



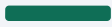
Support

Sub-4×4 matrices via padding without hardware duplication



Demonstrate

The design on a real, general-purpose application



Deploy

Synthesize and physically run on real FPGA hardware, not simulation alone



Characterize

Real timing, area, and power from actual synthesis reports

09 Work Completion / Implementation Status

Module	RTL	Testbench	Status
Processing element	Done	Done	Verified
Systolic array	Done	Done	Verified
Skew / de-skew buffer	Done	Done	Verified
Output processing (bias/ReLU)	Done	Done	Verified
Address generator (+padding)	Done	Done	Verified
Control unit (FSM, +weight reuse)	Done	Done	Verified
Top-level integration (+padding)	Done	Done	Verified (black-box)
Weight/activation/output memory	Done	—	Proven via top-level tests
FPGA deployment (UART, ZedBoard)	Done	Done	Verified on real hardware
Application: RGB channel extraction	Done	Done	Verified (sim + real hardware)
Synthesis (area/timing/power)	Done	—	Complete
Tiling (matrices > 4×4)	—	—	Designed, not implemented

11 of 12 tracked items fully complete and verified — the one remaining item (tiling) is scoped, designed, and documented for future work.

10 Results

Functional verification

Every module passed its self-checking testbench against a golden reference model.

- ✓ Known-case testing
- ✓ Identity passthrough
- ✓ Timing alignment regression
- ✓ Back-to-back matmuls
- ✓ Random full-system stress testing
- ✓ Padding correctness across all K_real values
- ✓ Poison-sensitivity testing

Every test category above ran to completion with zero unresolved failures across the full RTL implementation.

Top-level testing was black-box — driven only through external ports (start/done, memory read/write) — mirroring how a real host would use the accelerator.

10 Results

Synthesis results — Vivado 2020.1, target xc7z020clg484-1 (ZedBoard)

Target frequency	100 MHz
Timing	MET — WNS = +1.463 ns (max ≈117 MHz)
LUTs	1,694 / 53,200 (3.18%)
Registers	899 / 106,400 (0.84%)
Total on-chip power	0.121 W (0.015 W dynamic, 0.106 W static)
Routing	0 errors, 100% of nets fully routed
DRC	0 errors (1 informational PS7 warning)
I/O pins used	4 (clk, reset, UART RX, UART TX)

10 Results

Real cycle count — refined from synthesis-confirmed RTL

$4N - 2 = 14$ cycles (simplified theoretical model)

15 cycles — the real, synthesis-confirmed figure

- The FSM's phase transitions are sampled ONE cycle after `addr_done` asserts — not the same cycle — a real latency the idealized $4N-2$ formula doesn't capture.
- $\text{LOAD} = N+1 = 5$ cycles, $\text{COMPUTE} = N+1 = 5$ cycles, $\text{DRAIN} = 5$ cycles $\rightarrow$ 15 cycles total.

AT THE ACHIEVED 100 MHz

150 nanoseconds per matrix multiply

10 Results

Application — RGB channel extraction via matrix multiplication

- Each pixel $[R,G,B]$ is multiplied by a fixed diagonal selector matrix (padded 4×4 , $K_{\text{real}}=3$).
- The selector isolates one channel and zeroes the other two.
- Same selector matrix reused across every pixel batch in a channel — a real demonstration of weight-stationary reuse.

$$[R, G, B] \times \text{diag}(1,0,0) = [R, 0, 0]$$

$$[R, G, B] \times \text{diag}(0,1,0) = [0, G, 0]$$

$$[R, G, B] \times \text{diag}(0,0,1) = [0, 0, B]$$

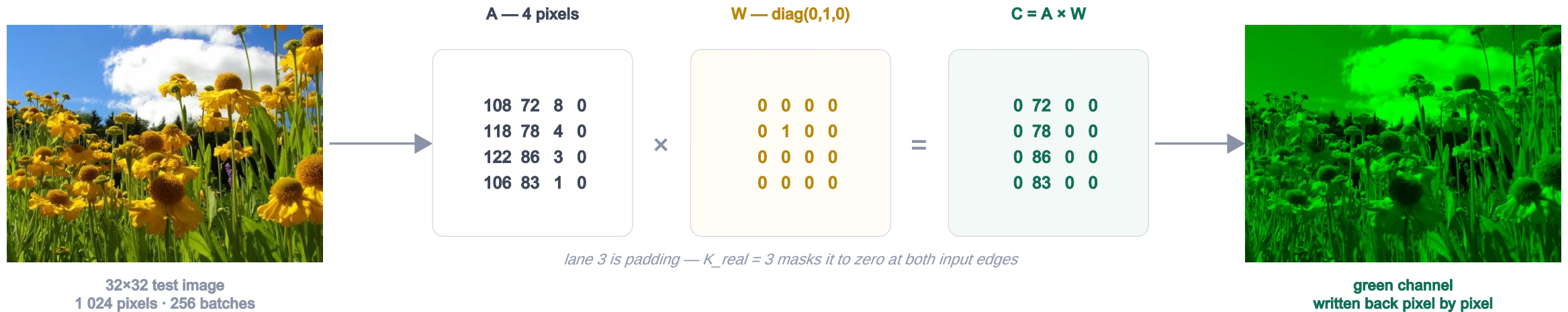
Verification: summing the three channel outputs back together ($R+G+B$) exactly reconstructs the original photo.



Original test photo

10 Results

Application pipeline — image → matrix → filter → image



1 image → matrix

Pixels are read four at a time. Each batch becomes one activation block — one pixel per row, [R, G, B] in lanes 0–2.

2 padding

Only three lanes carry data, so runtime $K_{\text{real}} = 3$ zeroes lane 3. The same synthesized hardware handles any size from 1x1 to 4x4.

3 filter

The selector matrix is loaded once and stays resident in the PEs. Every later batch in the channel reuses it — skip_load, 33.2% fewer cycles.

4 matrix → image

Each result row is written back as one filtered pixel. 256 batches rebuild one channel; the three channels sum back to the original photo.

Nothing in the hardware changes between the three channels — only the resident weight matrix does. That reuse is exactly what the weight-stationary dataflow was chosen for.

10 Results

Application output — real accelerator output, channel-split



Red channel



Green channel



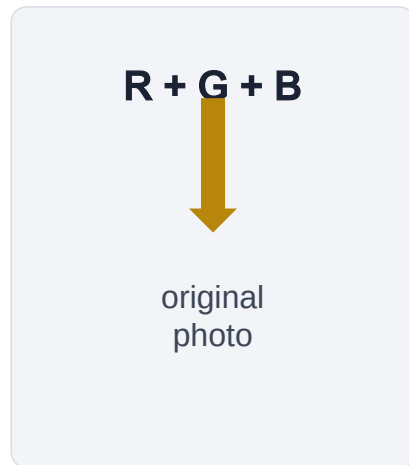
Blue channel

These images were produced by the actual accelerator — verified in simulation AND on real ZedBoard hardware.

Each channel required 256 accelerator runs (batches of 4 pixels) for this 32×32 test image — 768 runs total across all three channels.

10 Results

Correctness check — reconstructing the original



Reconstructed by summing the three real accelerator outputs — pixel-for-pixel match to the original, confirming the math is genuinely correct, not just plausible-looking.

10 Results

Weight-stationary reuse — measured efficiency

Worked example: a 32×32 test image — 1,024 pixels, 256 batches per channel.

WITHOUT REUSE

$$256 \times 15 = 3,840 \text{ cycles}$$

every batch reloads the same selector matrix

WITH REUSE (skip_load)

$$15 + 255 \times 10 = 2,565 \text{ cycles}$$

only the first batch loads; the other 255 skip straight to COMPUTE

1,275 CYCLES SAVED PER CHANNEL — 33.2% REDUCTION

Across all 3 channels: 11,520 → 7,695 cycles — 3,825 cycles saved for the full image.

10 Results

Real hardware deployment — ZedBoard

- Synthesized, implemented, and run on physical ZedBoard silicon — not simulation alone.
- Verified in stages: UART loopback (physical link only) → minimal single-batch identity test (full protocol + datapath) → complete RGB application on real hardware.
- Final working baud rate: 921,600 (upgraded from an initial 115,200 after confirming hardware stability).

REAL BUG #1

Built-in USB-UART port is hardwired to the Zynq's processor system — physically unreachable from FPGA fabric logic.

FIX:

Routed UART through an external USB adapter on a Pmod header instead.

REAL BUG #2

Reset button polarity mismatch — idle-low button vs. the design's active-low reset expectation.

FIX:

Diagnosed via systematic hardware debugging, fixed via signal inversion.

Both issues were found through disciplined, staged bring-up — never debugged blind against the full application.

11 Pros and Cons

PROS

- Very low resource utilization (3.18% LUTs) — substantial headroom to scale
- Very low power draw (0.121 W total)
- Timing met with real margin, not barely passing
- Genuinely reconfigurable: array size, data width, bias/ReLU, matrix size — all parameterizable
- Weight-stationary reuse reduces redundant reloading — 33.2% cycle savings measured
- Verified at both simulation AND real-hardware level
- Adversarial verification: poison-data testing, not just nominal cases
- Bottom-up methodology caught timing bugs early, before system integration

CONS

- Fixed maximum matrix size (4×4) without tiling
- UART host link is a bandwidth bottleneck for large real-world datasets (host-interface limit, not compute limit)
- Only one dataflow mode implemented and verified in hardware
- Memory modules lack dedicated unit-level testbenches (proven only via top-level integration)
- No automated tiling means larger matrices currently require manual decomposition

12 Future Scope

Including applications

TECHNICAL EXTENSIONS

- Tiling support for matrices larger than 4×4 (architecture already designed and documented)
- Replace the UART host link with an AXI/PS-integrated interface for far higher real-world throughput
- Scale array size beyond 4×4 for higher-throughput workloads
- Explore alternative dataflow modes as switchable datapaths alongside the existing systolic mode
- Automated pipeline for streaming larger images directly from an SD card via the Zynq PS

APPLICATION DIRECTIONS

- Small neural-network inference layers — output-processing's bias/ReLU support was built with this in mind
- Other convolution-based image filters (edge detection, blur) expressible as matrix multiplication
- General-purpose linear algebra acceleration for power-constrained embedded/edge contexts
- Real-time sensor data processing where low power and predictable latency both matter

Motivated directly by this project's measured 0.121 W power result — a genuinely low-power foundation for edge deployment.

13 Paper Publication

STATUS: Paper in preparation — not yet submitted anywhere.

Why this project is publishable

- The verification methodology — bottom-up, self-checking, adversarial poison-data testing — is substantial and well-documented.
- Real hardware deployment results (synthesis + physical FPGA run) go beyond simulation-only student projects.
- The measured weight-reuse efficiency result (33.2% cycle reduction) is a concrete, quantifiable contribution.

Potential venues

- Student / undergraduate research conferences
- University-level VLSI or embedded systems symposia
- IEEE student chapter publications

14 Conclusion

- A fully verified, weight-stationary systolic array matrix multiplication accelerator — taken from RTL design through simulation, synthesis, and real FPGA deployment.
- Not just “works in simulation” — confirmed real timing, area, and power numbers, AND a working real-hardware demonstration with an actual application.
- Tiling is a well-understood, documented future extension — deliberately deferred to prioritize verification depth on the implemented core.
- The full journey, end to end: architecture decisions, bottom-up RTL, exhaustive self-checking verification, synthesis, and physical silicon.

3.18%

LUT usage

0.121W

power

100 MHz

timing met

15

cycles/matmul

Thank you.